Week1_Individual_Assignment_Sulaiman (Stan)_Abu Romman

Section 1: What is Conceptual Thinking?

Conceptual thinking is the ability to look at a problem or situation and understand the bigger picture behind it instead of just focusing on the small details. It is about recognizing patterns, relationships, and ideas that connect different parts of a problem. In software development, this means not only viewing a single feature but thinking about how the entire system should work together for the users. It assists in planning solutions that are useful and clear, not just rapid fixes.

This type of thinking is essential nowadays because most problems are complex and involve many people, systems, and goals. For instance, apps like Uber or Google Maps work efficiently because their creators thought about the full experience — not just how to book a ride or see a map, but how drivers, routes, payments, and user safety all connect into one smooth system simultaneously. Another example is Amazon: its success comes from a vivid concept of how products, sellers, warehouses, and customers interact in one place.

Using conceptual thinking means you plan ahead, spot potential issues early, and build systems that could genuinely solve the right problem instead of making it more complicated.

Section 2: Phases of the Software Development Life Cycle (SDLC)

The Software Development Life Cycle, or SDLC, is the process used to plan, build, test, and maintain software. It includes several phases that keep a project organized from the beginning to the end. The common phases are Planning, Analysis, Design, Implementation, Testing, Deployment, and Maintenance. Each one has its own goal and makes the next step possible.

The Planning phase defines what the problem is and what the software should achieve. The Analysis phase studies user needs and gathers requirements. The Design phase decides how the system will look and work technically. The Implementation phase is where coding occurs, followed by Testing to make sure everything works properly. Deployment releases the finished product to users, and Maintenance keeps it operating smoothly post-release.

The Planning and Analysis phases are the most critical because they build the foundation for the rest. If the problem or requirements are not initially clear, the whole project can go in the wrong direction. These phases help developers avoid mistakes, save time, and make sure the final system meets the real needs of users.

Section 3: Waterfall vs Iterative Development:

Waterfall and Iterative are two different models for developing software. The Waterfall model moves in one straight direction: you finish one phase completely before starting the next. It is simple and easy to manage, but not flexible. Once you move forward, it is difficult to go back and modify something. It works best for small projects with clear and stable requirements, such as creating a calculator app or an internal company tool.

The Iterative model, on the other hand, builds the system in small cycles. Developers plan, design, code, and test small parts, then enhance them through feedback. It is more flexible and fits modern projects where requirements can vary. For example, large web platforms or mobile apps like Instagram use Iterative or Agile methods to release updates gradually and adjust to user feedback.

The core difference is that Waterfall focuses on control and structure, while Iterative focuses on learning and enhancements. Each has its place: Waterfall for predictable projects, and Iterative for dynamic ones that evolve.

Section 4: Research Company Methodologies:

Different companies use different software development methodologies depending on their needs and goals. For instance, Microsoft mainly uses Agile and Scrum for most of its projects. These methods allow small teams to work on short cycles, known as sprints, and deliver new updates often. This helps Microsoft enhance its products like Windows 11 or Office 365 regularly, based on user feedback and performance reports. Agile makes it easier for them to stay competitive and adapt rapidly to new technologies.

Another example is NASA, which often uses the Waterfall or V-Model approach for its space and research projects. In such systems, safety and reliability are more essential than speed or flexibility. Once the design is locked, it must be tested carefully and follow strict documentation before proceeding to the next stage. This ensures that every part of the system is verified and nothing fails in space missions.

Both companies show how methodology choices depend on the kind of project. Agile helps with fast-moving, constantly changing environments, while Waterfall is suitable for highly critical systems that require precision and full documentation.

Assignment for Week 2: **Welcome to the World of Objects**
Part 2: Theory and Research Questions (group)

1. Glossary

Term	Definition
User (Customer)	A person who visits the website or app to buy tickets for events.
Organizer	A person or company that creates and manages events.
Event	A concert, match, or show that users can buy tickets for.
Ticket	A digital pass that allows a user to attend an event.
Order	A purchase that contains one or more tickets.
Payment	The money transaction made when buying tickets.
E-Ticket	A digital version of the ticket with a QR code.
Refund	Returning money to the user when an order is cancelled.
Venue Staff	People who scan tickets at the event entrance.

2. Object-Oriented Analysis (OOA)

Object	Attributes	Responsibilities
User	name, email, password	Browse events, buy tickets, request refunds
Organizer	name, contact info	Create and manage events
Event	title, date, venue, price	Show event info, hold ticket data
Ticket	ticketID, eventID, status	Represent entry to an event
Order	orderID, userID, total	Store purchase details
Payment	paymentID, method, status	Handle payment confirmation
Venue Staff	staffID	Scan and validate tickets

3. Requirements (FURPS)

Functional Requirements (User Stories):

1. As a user, I want to search for events so I can find something to attend.
2. As a user, I want to buy tickets online so I can go to the event.
3. As an organizer, I want to add new events so users can buy tickets.
4. As venue staff, I want to scan tickets to check if they are valid.

Use Case Description – Buy Ticket:

- Actors: User, Payment Gateway
- Preconditions: User is logged in and tickets are available.
- Main Flow:
    1. User selects event and ticket type.
    2. User enters payment details.
    3. System confirms payment and sends an e-ticket.
- Alternative Flow: Payment fails → User is notified.
- Postconditions: Ticket is created and sent to the user.

Category	Requirement
Usability	Simple and mobile-friendly interface.
Reliability	System should work 24/7 with minimal downtime.
Performance	Pages should load within 2 seconds.
Supportability	System should keep logs for errors and transactions.

4. Use Case Diagram (Description for Drawing)

System: Online Event Ticketing System
Actors: User, Organizer, Payment Gateway, Venue Staff
Use Cases: Browse Events, Buy Ticket, Make Payment, Download Ticket, Scan Ticket

Relationships:

- User → Browse Events, Buy Ticket, Download Ticket
- Organizer → Add/Manage Events
- Venue Staff → Scan Ticket
- Payment Gateway ↔ Make Payment

Business Case: Online Event Ticketing System

1. Use Case Diagram (Optimized)

System Name: Online Event Ticketing System

Actors:

- User (Customer)
- Organizer
- Payment Gateway
- Venue Staff

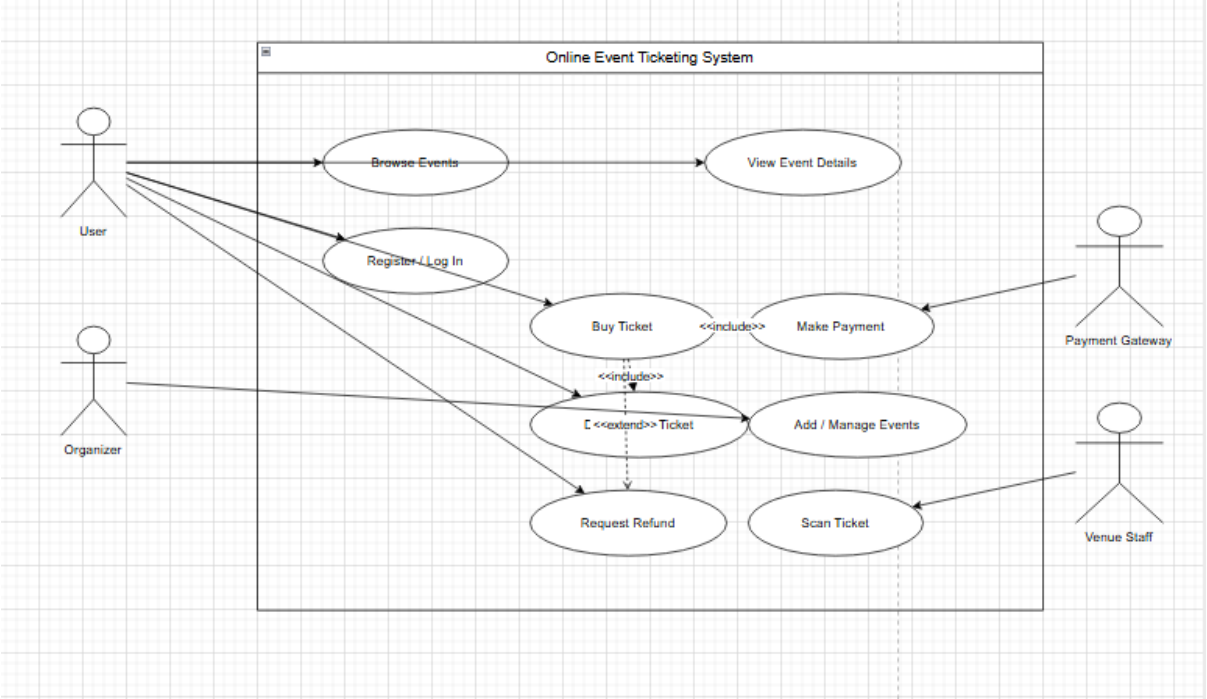
Main Use Cases:

- Register / Log In
- Browse Events
- View Event Details
- Buy Ticket
- Make Payment
- Download E-Ticket
- Request Refund
- Add / Manage Events
- Scan Ticket

Relationships:

- Buy Ticket → includes → Make Payment
- Buy Ticket → includes → Download E-Ticket
- Buy Ticket → extends → Request Refund
- Organizer → includes → Add / Manage Events
- Venue Staff → includes → Scan Ticket

System Boundary:
All user and organizer actions are inside the Online Event Ticketing System. The Payment Gateway is an external system.



2. Story Mapping

Goal: Allow users to easily buy, manage, and use event tickets.

Activity	Epic (Group of Stories)	User Stories
Browsing Events	Search & View	- As a user, I want to search for events so that I can find one I like.- As a user, I want to view event details so that I can decide whether to attend.
Buying Tickets	Purchase Process	- As a user, I want to buy tickets online so that I can attend an event.- As a user, I want to receive an e-ticket so that I can show it at the entrance.
Payments	Secure Transactions	- As a user, I want to make safe payments so that my personal data is protected.- As a user, I want to request a refund so that I can cancel an order if necessary.
Managing Events	Organizer Actions	- As an organizer, I want to create and edit events so that users can buy tickets.
Event Entry	Validation	- As venue staff, I want to scan tickets so that I can verify their validity at the entrance.

3. Wireframes

Wireframe 1 – Home Page (Browse Events)

- Header: Home | Events | Login | My Tickets
- Search bar: “Search for events…”
- Event cards: image, title, date, price, and “Buy Ticket” button

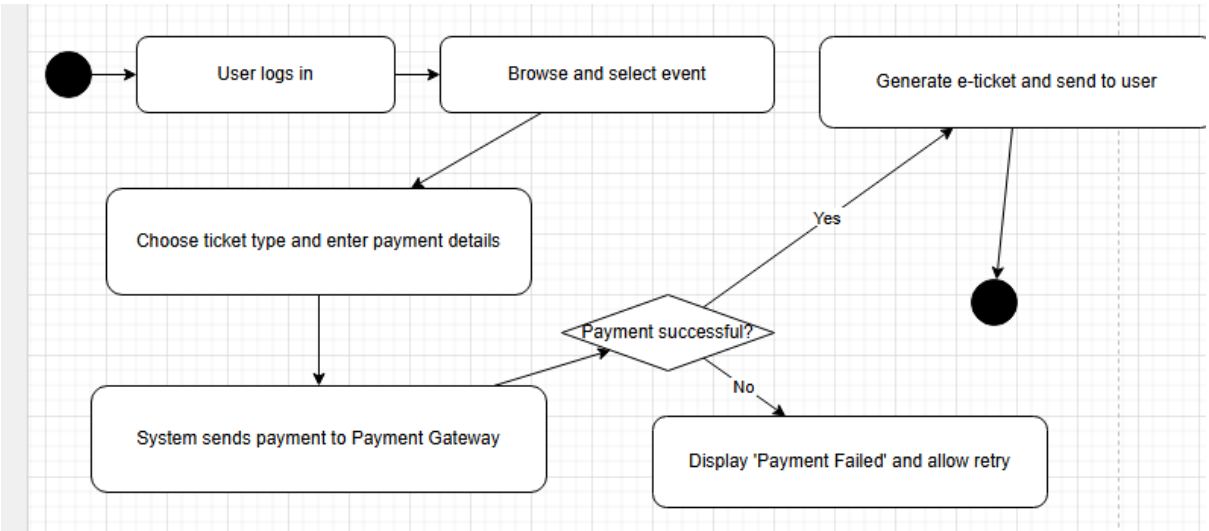
Wireframe 2 – Event Detail / Booking Page

- Event Title, Description, Date, Venue, Price
- Ticket Type selector (Standard, VIP)
- Quantity selector
- “Buy Ticket” button → opens Payment form

- Payment form: Card details and “Confirm Payment” button
- Confirmation message: “Payment successful. Your ticket has been sent via email.”

4. Activity Diagram (Buy Ticket Flow)

Flow description:
Start → User logs in → User browses and selects event → User chooses ticket type and enters payment details → System sends payment info to Payment Gateway → Decision: Payment successful? → If Yes: System generates e-ticket and sends to user → End. If No: Display “Payment Failed” and allow retry.



5. Reflection

After learning more about use case diagrams, we optimized our diagram by correctly applying “include” and “extend” relationships and clarifying the system boundary. The story map helped us structure our user stories logically, while the activity diagram clearly visualized the flow of the main “Buy Ticket” use case. Overall, the system is now better structured and easier to understand.

Week 4 Assignment – Tell The Story Of Objects

Business Case: Online Event Ticketing System
Group Members: Aeen, Stan

Part 1: Group Task – Conceptual Class Diagram Elaboration

Introduction

In Week 3, we created a first version of our domain model for the Online Event Ticketing System. For Week 4, we refined it by keeping only the essentials, adding clear attributes, and specifying multiplicities and business rules. We kept it conceptual (the “what”), not technical (the “how”), so it’s easy to read and use in class.

Conceptual Classes

- **Customer** — customerId, name, email, phoneNumber, address
- **Order** — orderId, orderDate, totalAmount, status, paymentMethod
- **Ticket** — ticketId, price, seatNumber?, qrCode, status
- **Event** — eventId, title, date, category, description
- **Venue** — venueId, name, location, capacity, type
- **Payment** — paymentId, amount, status, method, paymentDate
- **Organizer** — organizerId, name, contactInfo, organizationName, role

Associations and Multiplicities

- Customer **1..*** Order
- Order **1..*** Ticket
- Ticket **1..1** Event
- Event **1..1** Venue
- Organizer **1..*** Event
- Order **1..1** Payment

These links describe the core flow: organizers create events at venues; customers place orders, which contain tickets for events; each order has one payment.

Business Rules (constraints)

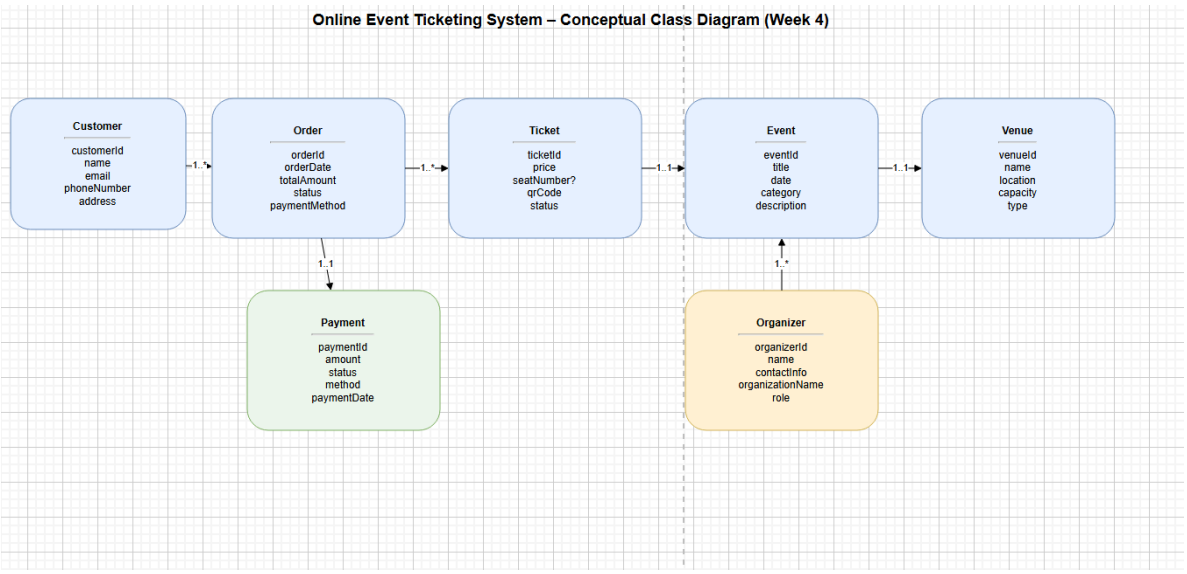
1. An order must be paid before tickets are confirmed (Payment.status = “Paid”).
2. Venue.capacity limits the total number of tickets sold for an event date.
3. Each ticket is tied to exactly one event; seatNumber is optional for standing events.
4. An order has one payment; refunds (if needed) are handled later and are out of scope for Week 4.
5. Customer contact details (email/phone) are required for delivery of e-tickets.

Explanation

We simplified the Week 3 model to keep only the most important concepts so the diagram stays readable. **User** was narrowed to **Customer** and **Organizer** because those are the two roles we actually need here: customers buy tickets, organizers create events. **Event** is linked to **Venue** to capture where the event happens and to support a capacity constraint. **Ticket** connects orders to events (one ticket = one event occurrence) and optionally includes a seat number for seated shows. **Order** and **Payment** are modeled one-to-one to make the transaction flow easy to understand and track.

We assigned a small, meaningful set of attributes to every class so the model looks realistic without dropping into implementation details. The multiplicities reflect real situations (a customer can have many orders; an order can contain many tickets; each ticket is for one event). The business rules make sure the model can’t describe impossible states (like selling more tickets than the venue can handle, or un-paid orders generating valid tickets).

Overall, this conceptual class diagram is a clear continuation of our Week 3 work. It keeps the domain story simple, covers the main objects and their relationships, and gives us a solid base for Week 5 when we’ll go deeper into more advanced modeling.



Week 5 – Tell the Story of Objects (Part 2)

Business Case: Online Event Ticketing System (Spot Saver)
Group Members: Aeen, Stan

Introduction

In Week 3 we first learned about domain models and built a basic version of our system. During Week 4 we refined that diagram by focusing on clearer class names, key attributes, and logical relationships. Now, in Week 5, we continued that process by applying the new theory about conceptual class diagrams. We improved the realism of our model while still keeping it conceptual and readable for a first-year level. The goal was to “tell the story” of how objects in our Online Event Ticketing System relate to each other in a real situation.

Updated Conceptual Model

The refined model still centers on seven main classes — **Customer, Order, OrderLine, Ticket, TicketType, Event, Venue** — and also includes **Payment** and **Organizer** as supporting concepts.
Each class has meaningful attributes, for example:

- *Customer:* customerId, name, email, phoneNumber
- *Order:* orderId, orderDate, totalAmount, status (OrderStatus)
- *Ticket:* ticketId, qrCode, status (TicketStatus), seatNumber?
- *Event:* eventId, title, date, category (EventCategory)
- *Venue:* venueId, name, location, capacity

The **associative class OrderLine** connects *Order* and *TicketType*. It represents each line item inside an order, with attributes such as *quantity*, *unitPrice*, and *lineTotal*. This small addition fixes the common problem of representing multiple ticket types within a single order.

We also introduced **enumeration classes** to make data values consistent:

- *OrderStatus* → Pending | Paid | Cancelled
- *TicketStatus* → Reserved | Issued | Cancelled
- *PaymentMethod* → Card | Bank Transfer
- *EventCategory* → Concert | Sport | Theatre

These enumerations limit invalid entries and clarify the state of each object without needing free-text fields.
Each relationship in the diagram uses clear multiplicities (for example, *one Customer can place many Orders*, *each Order has one Payment*).

Business Rules and Constraints

To make the model realistic, we added a few simple but useful rules as yellow "notes" in the diagram:

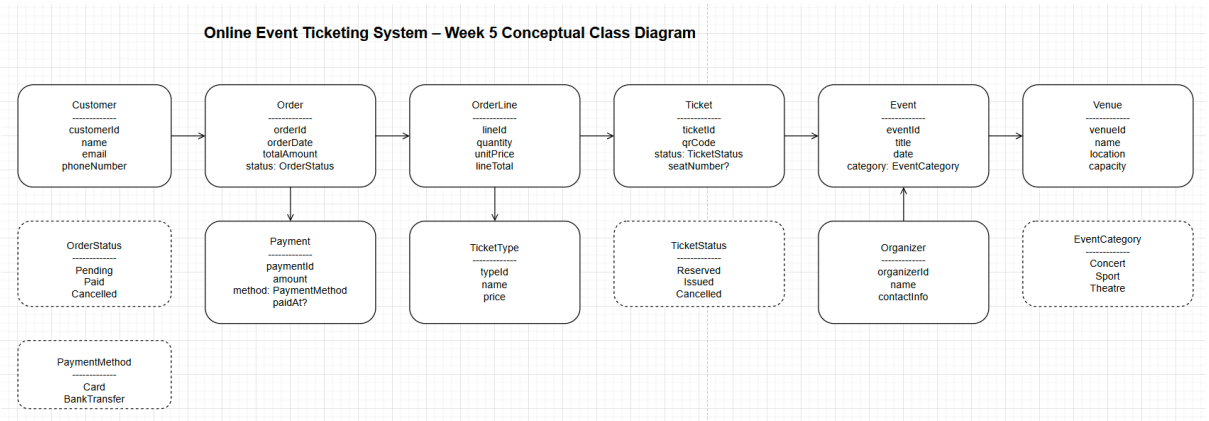
1. **BR1:** Tickets sold for an Event must be less than or equal to the Venue capacity.
2. **BR2:** An Order only becomes "Paid" when a Payment method is selected and a payment date exists.
3. **BR3:** Order.totalAmount = sum of all OrderLine.lineTotal values.

These rules make it clear that the diagram isn't just a drawing — it also describes logical business behaviour.

Reflection and Conclusion

Compared to Week 4, the new version feels much closer to a working concept for an actual ticketing platform. The model now shows how every ticket purchase moves from a Customer to an Order, through Payment, and finally to an Event at a Venue created by an Organizer. The addition of enumerations and an associative class makes the structure cleaner and prevents data errors. This conceptual class diagram forms a strong foundation for the next phases of the SDLC, where we will later convert it into a design-level UML

diagram or database schema. For now, it clearly communicates **what** the system contains and **how** the objects relate — the core goal of conceptual modeling.



Week1_Individual_Assignment_Sulaiman (Stan)_Abu Romman

Section 1: What is Conceptual Thinking?

Conceptual thinking is the ability to look at a problem or situation and understand the bigger picture behind it instead of just focusing on the small details. It is about recognizing patterns, relationships, and ideas that connect different parts of a problem. In software development, this means not only viewing a single feature but thinking about how the entire system should work together for the users. It assists in planning solutions that are useful and clear, not just rapid fixes.

This type of thinking is essential nowadays because most problems are complex and involve many people, systems, and goals. For instance, apps like Uber or Google Maps work efficiently because their creators thought about the full experience — not just how to book a ride or see a map, but how drivers, routes, payments, and user safety all connect into one smooth system simultaneously. Another example is Amazon: its success comes from a vivid concept of how products, sellers, warehouses, and customers interact in one place.

Using conceptual thinking means you plan ahead, spot potential issues early, and build systems that could genuinely solve the right problem instead of making it more complicated.

Section 2: Phases of the Software Development Life Cycle (SDLC)

The Software Development Life Cycle, or SDLC, is the process used to plan, build, test, and maintain software. It includes several phases that keep a project organized from the beginning to the end. The common phases are Planning, Analysis, Design, Implementation, Testing, Deployment, and Maintenance. Each one has its own goal and makes the next step possible.

The Planning phase defines what the problem is and what the software should achieve. The Analysis phase studies user needs and gathers requirements. The Design phase decides how the system will look and work technically. The Implementation phase is where coding occurs, followed by Testing to make sure everything works properly. Deployment releases the finished product to users, and Maintenance keeps it operating smoothly post-release.

The Planning and Analysis phases are the most critical because they build the foundation for the rest. If the problem or requirements are not initially clear, the whole project can go in the wrong direction. These phases help developers avoid mistakes, save time, and make sure the final system meets the real needs of users.

Section 3: Waterfall vs Iterative Development:

Waterfall and Iterative are two different models for developing software. The Waterfall model moves in one straight direction: you finish one phase completely before starting the next. It is simple and easy to manage, but not flexible. Once you move forward, it is difficult to go back and modify something. It works best for small projects with clear and stable requirements, such as creating a calculator app or an internal company tool.

The Iterative model, on the other hand, builds the system in small cycles. Developers plan, design, code, and test small parts, then enhance them through feedback. It is more flexible and fits modern projects where requirements can vary. For example, large web platforms or mobile apps like Instagram use Iterative or Agile methods to release updates gradually and adjust to user feedback.

The core difference is that Waterfall focuses on control and structure, while Iterative focuses on learning and enhancements. Each has its place: Waterfall for predictable projects, and Iterative for dynamic ones that evolve.

Section 4: Research Company Methodologies:

Different companies use different software development methodologies depending on their needs and goals. For instance, Microsoft mainly uses Agile and Scrum for most of its projects. These methods allow small teams to work on short cycles, known as sprints, and deliver new updates often. This helps Microsoft enhance its products like Windows 11 or Office 365 regularly, based on user feedback and performance reports. Agile makes it easier for them to stay competitive and adapt rapidly to new technologies.

Another example is NASA, which often uses the Waterfall or V-Model approach for its space and research projects. In such systems, safety and reliability are more essential than speed or flexibility. Once the design is locked, it must be tested carefully and follow strict documentation before proceeding to the next stage. This ensures that every part of the system is verified and nothing fails in space missions.

Both companies show how methodology choices depend on the kind of project. Agile helps with fast-moving, constantly changing environments, while Waterfall is suitable for highly critical systems that require precision and full documentation.

Week2_Individual_Assignment_Sulaiman (Stan)_Abu Romman

Section 1: What is a Use Case Diagram?

A use case diagram is a simple visual representation that demonstrates how users interact with a system. It describes what actions can be performed and who performs them. The focus is not on how the system is built internally but on what the system does from a user's point of view. A use case diagram helps identify the main features of a system and the interactions between different types of users and the system itself. It is typically created early in a project to capture the overall functionality in a clear and concise manner.

Section 2: Components of a Use Case Diagram:

A use case diagram includes three main components: actors, use cases, and relationships. Actors represent people, organizations, or external systems that interact with the application. Use cases represent the actions or functions the system can perform, such as placing an order, logging in, or making a payment. Relationships are the links that connect actors with their use cases, showing who implements which actions. There are also special types of relationships, such as "include" and "extend," which show dependencies or optional behaviors between different use cases.

Section 3: Purpose of a Use Case Diagram:

The core purpose of a use case diagram is to provide a clear understanding of how a system behaves from the user's perspective. It helps communicate the system's functionality between developers, designers, and stakeholders. By focusing on what users can do, it ensures that all important requirements are captured early in the design phase. Use case diagrams are useful for planning, identifying missing features, and confirming that the system will meet user expectations before the development process begins.

Week3_Individual_Assignment_Sulaiman (Stan)_Abu_Romman

Section 1: Introduction

This individual task is part of Week 3's "Welcome to the World of Objects" assignment. The goal is to understand the concept of a domain model and to create a simple domain model for a selected business case. I chose the Online Food Ordering System, which enables customers to browse menus, place orders, and receive deliveries.

Section 2: What is a Domain Model?

A domain model, also referred to as a class diagram, is a visual representation of the main concepts, objects, and relationships in a system. It helps describe what exists in the system, such as customers, orders, and products, without focusing on how it is implemented in code. Each concept is represented as a class, which includes its attributes and associations with other classes.

Section 3: Why is the Domain Model used?

A domain model is used to help understand, analyze, and communicate the structure of a system before programming it. It provides a shared understanding between developers, designers, and stakeholders. It also assists in identifying what data will be needed, how different parts connect, and what relationships exist in the system. In short, it is a planning tool that makes later stages of software design easier, simpler, and more accurate.

Section 4: Typical Elements of a Domain Model

A domain model typically includes the following elements:

1. Classes – the main entities or objects in the system, e.g., Customer, Order, Restaurant.
2. Attributes – the pieces of information each class stores, e.g., name, email, or price.
3. Associations – relationships between classes, e.g., Customer places Order.

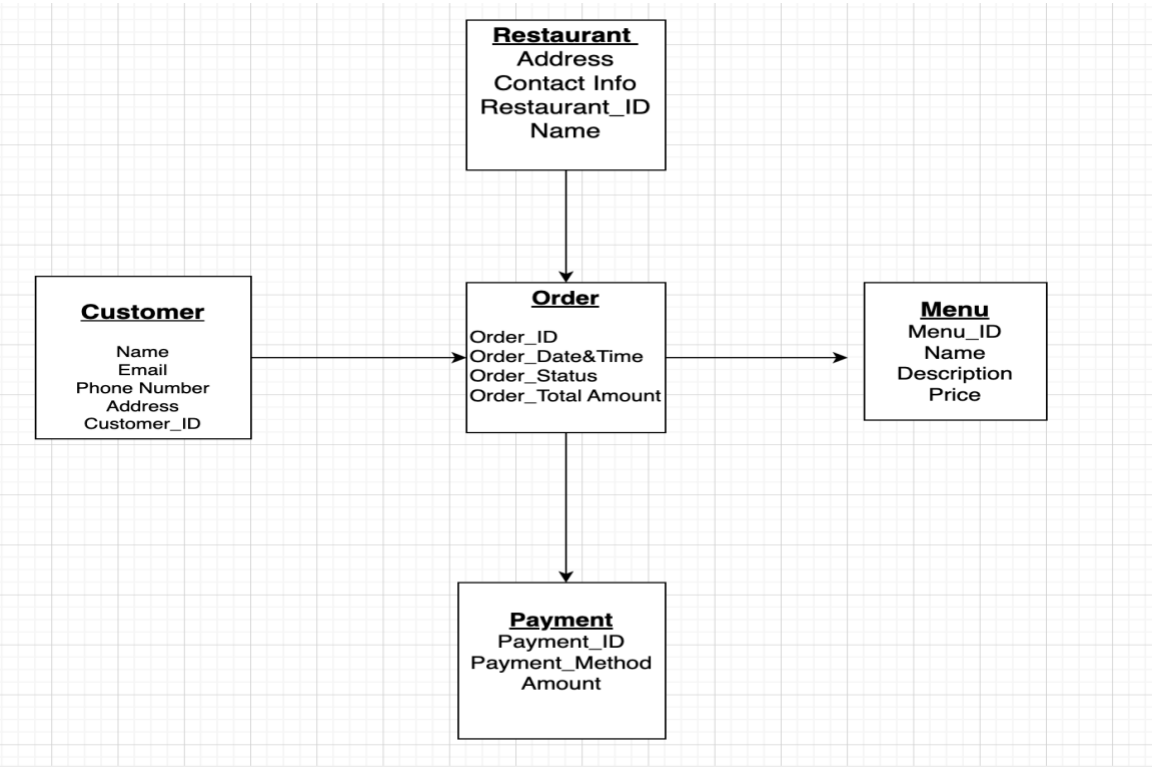
4. Multiplicities – how many objects are connected, e.g., one Customer can place many Orders.

Section 5: Example: Stomach Saver (Online Food Ordering System)

In my business case, the system is called Stomach Saver. It is an online food ordering platform that allows customers to browse restaurant menus, place food orders, and receive deliveries at their homes, workspaces, or anywhere they like. The idea behind Stomach Saver is to make ordering food quick, reliable, and easy for everyone.

The main classes in the Stomach Saver system are Customer, Restaurant, Item menu, Order, Order Item, Delivery Staff, and Payment. Each class represents a different part of how the system functions. Customers can browse menus and place orders. Restaurants prepare the food and update menu items. Delivery staff members deliver the orders to customers, and each order has a connected payment record.

Below is my first draft of a simple domain model for Stomach Saver, showing the relationships between these classes.



This model represents the key data structure of the system before moving to implementation.

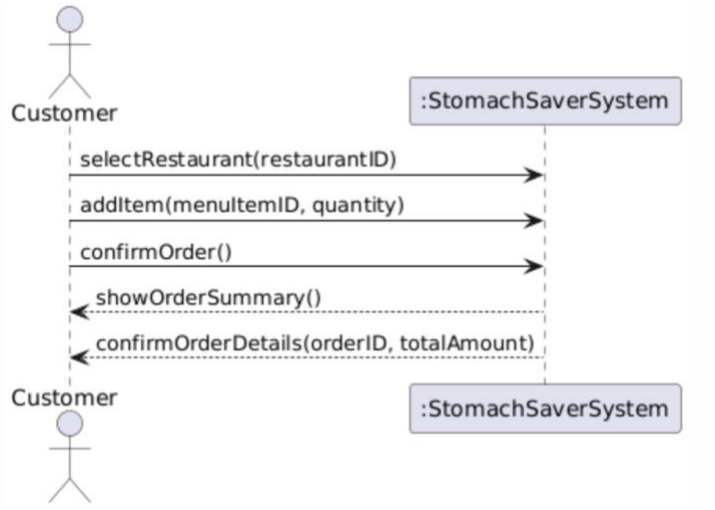
Group Assignment: From Analysis to Design
Stomach Saver – Online Food Ordering System
Prepared by: Sulaiman (Stan) Abu Romman

Part 1 – System Sequence Diagram (SSD)

A System Sequence Diagram (SSD) is a diagram that shows how an external actor interacts with the system as a whole. The system is treated as a black box, meaning that internal objects and implementation details are not shown. SSDs focus on what the system does rather than how it is implemented internally. They are derived from use case descriptions and help move from analysis to design.

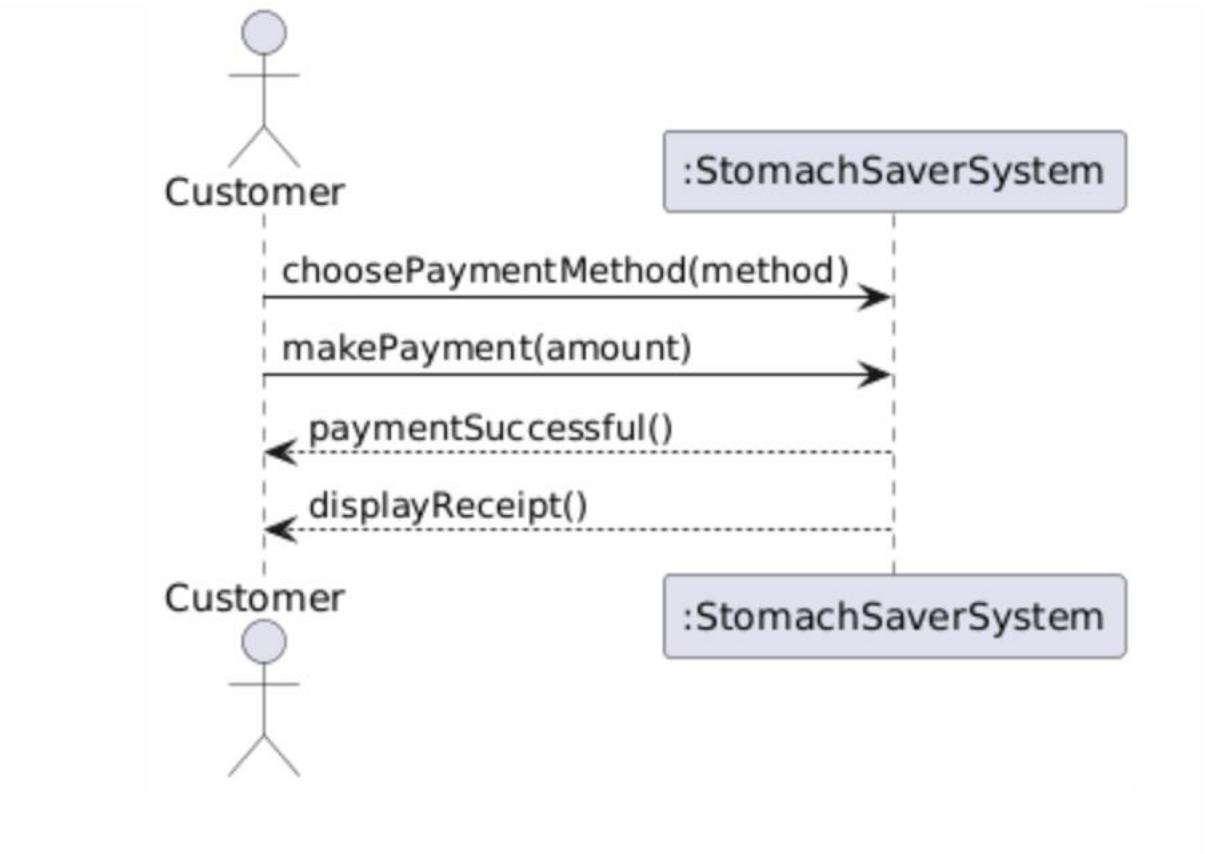
SSD 1 – Place Food Order (Stomach Saver)

This System Sequence Diagram describes how a customer places a food order using the Stomach Saver system. The customer selects a restaurant, adds menu items to the order, and confirms the order. The system then responds by displaying an order summary and confirming the order details.



SSD 2 – Make Payment (Stomach Saver)

This diagram illustrates how a customer completes the payment process. The customer selects a payment method and makes a payment. The system processes the payment and confirms it by displaying a receipt.



Part 2 – Operation Contracts (OC)

Operation Contracts describe how the internal state of the system changes after an operation is executed. They focus on object creation, associations, and attribute changes.

Operation Contract – confirmOrder()

Preconditions:
A Customer instance exists.
An Order instance exists and is not yet confirmed.

Postconditions:
The Order was confirmed.
The Order status was set to Confirmed.
The Order was linked to the Customer.
The total amount was calculated.

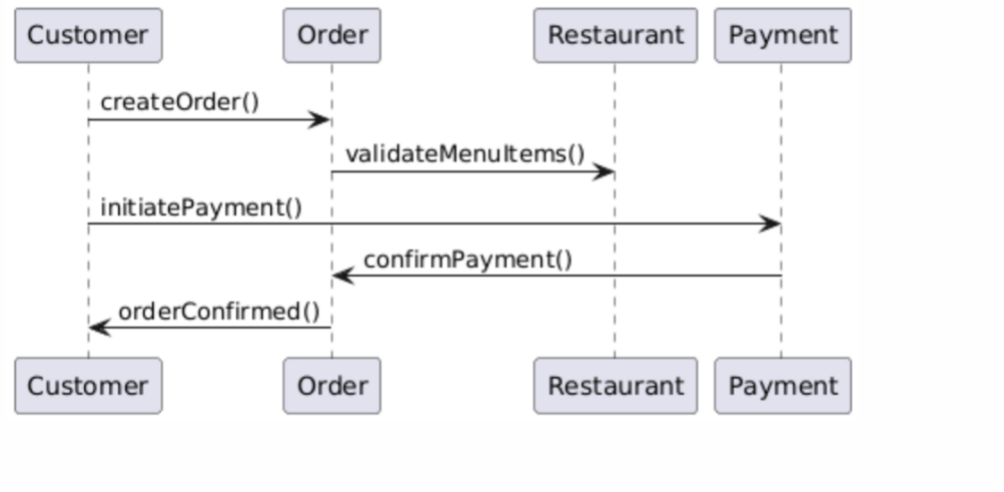
Operation Contract – makePayment(amount)

Preconditions:
A confirmed Order exists.
A Customer exists.

Postconditions:
A Payment instance was created.
The Payment was linked to the Order.
The Order status was updated to Paid.

Part 3 – Sequence Diagram

A Sequence Diagram shows how objects communicate with each other over time and is used to describe internal system behavior.



Week7_individual_assignment_Sulaiman (Stan) Abu Romman

Part 3 – Individual Research Task: Sequence Diagram

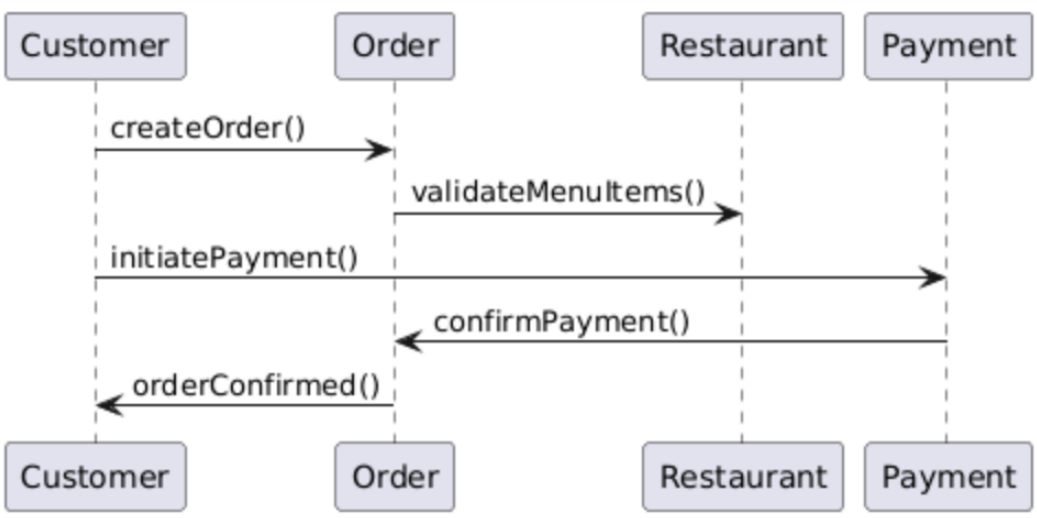
A Sequence Diagram is a UML interaction diagram that shows how objects communicate with each other over time. It focuses on the order in which messages are sent between objects during a specific scenario. Unlike a System Sequence Diagram, which treats the system as a black box, a sequence diagram shows the internal objects of the system and how they collaborate to perform a task.

The main components of a sequence diagram are objects or actors, lifelines, messages, and activation bars. Objects are placed horizontally at the top of the diagram, while time flows vertically from top to bottom. Messages between objects are represented by arrows, showing the direction and sequence of communication.

The purpose of a sequence diagram in software development is to support the design phase by explaining how the system works internally. It helps developers understand object responsibilities, message flow, and interaction logic before implementation. Sequence diagrams are especially useful for translating use cases into detailed system behavior.

Application – Sequence Diagram for Stomach Saver

The following sequence diagram represents the internal process of placing a food order in the Stomach Saver system. It shows how the customer interacts with system objects and how these objects collaborate to complete the order and payment process.


Customer
Order
Restaurant
Payment
createOrder()
validateMenuItems()
initiatePayment()
confirmPayment()
orderConfirmed()
Customer
Order
Restaurant
Payment

Week8_group&indivisual_assignment__Sulaiman (Stan) Abu Romman

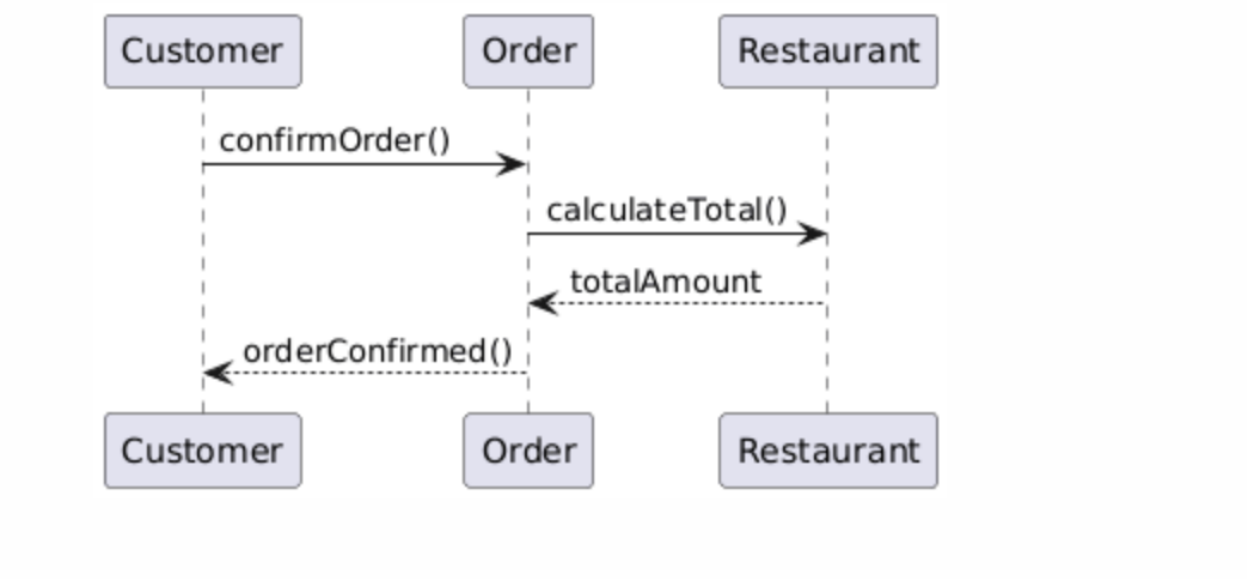
Part 1 – Sequence Diagrams (SD)

A Sequence Diagram is an interaction diagram used in the design phase of software development. It shows how objects within a system communicate with each other over time by sending messages. Unlike a System Sequence Diagram, which treats the system as a black box, a Sequence Diagram shows the internal objects and their interactions.

Sequence Diagrams are created based on Operation Contracts. Each operation defined in an Operation Contract is expanded into a Sequence Diagram that shows how the operation is executed internally.

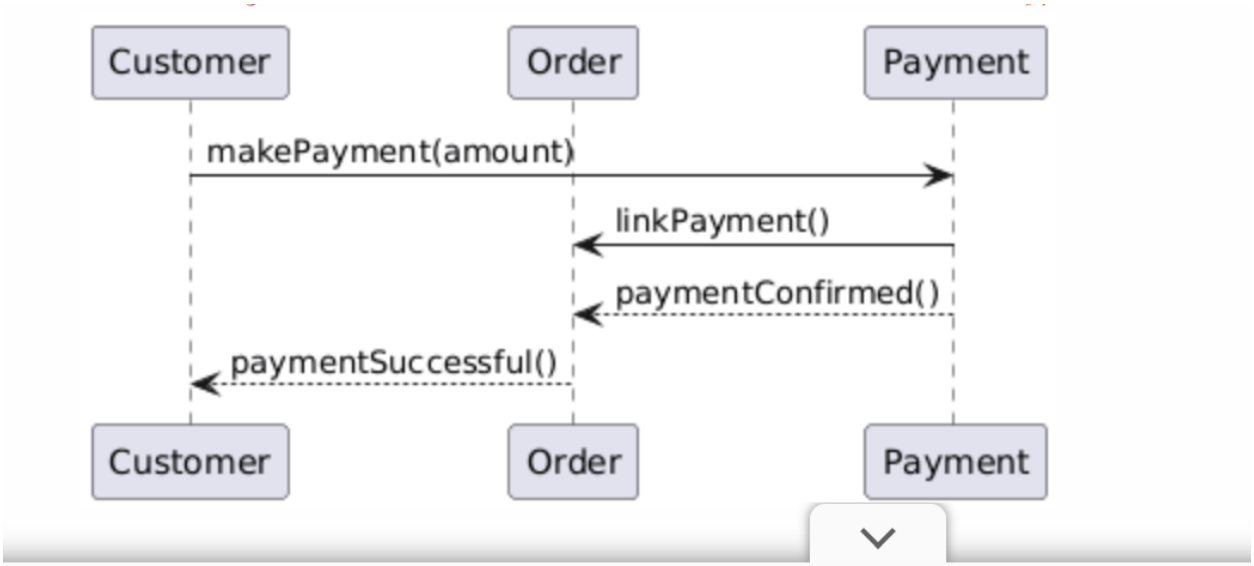
SD 1 – confirmOrder() – Stomach Saver

This sequence diagram represents how the system internally handles the confirmOrder operation in the Stomach Saver system. It shows how objects collaborate to confirm an order and calculate the total amount.



SD 2 – makePayment(amount) – Stomach Saver

This sequence diagram illustrates the internal execution of the makePayment operation. It shows how the payment is created, linked to the order, and confirmed.



Week8_individual_assignment_Sulaiman (Stan) Abu Romman

A Design Class Diagram (DCD) is a diagram that shows the classes of a system during the design phase, including their attributes, methods, and relationships. Unlike a domain model, which focuses on conceptual objects, a DCD is closer to implementation and includes method signatures derived from sequence diagrams.

A Design Class Diagram is directly linked to Sequence Diagrams. The messages shown in sequence diagrams become methods in the classes of the DCD. Each object interaction in an SD helps identify responsibilities and operations that must be implemented.

The purpose of a DCD is to translate system behavior into a structured class design that can be implemented in code. It helps developers understand class responsibilities, method definitions, and relationships before programming begins.

Draft Design Class Diagram – Stomach Saver

The following draft Design Class Diagram represents the main classes involved in the Stomach Saver system. The diagram includes classes such as Customer, Order, Payment, and Restaurant, along with their key attributes and methods derived from the sequence diagrams.


Customer
customerID
name
email
1
0..*
Order
orderID
status
totalAmount
confirmOrder()
1
1
1
1
Restaurant
restaurantID
name
calculateTotal()
Payment
paymentID
amount
makePayment()

Week9_group&indivisual_assignment_Sulaiman (Stan) Abu Romman

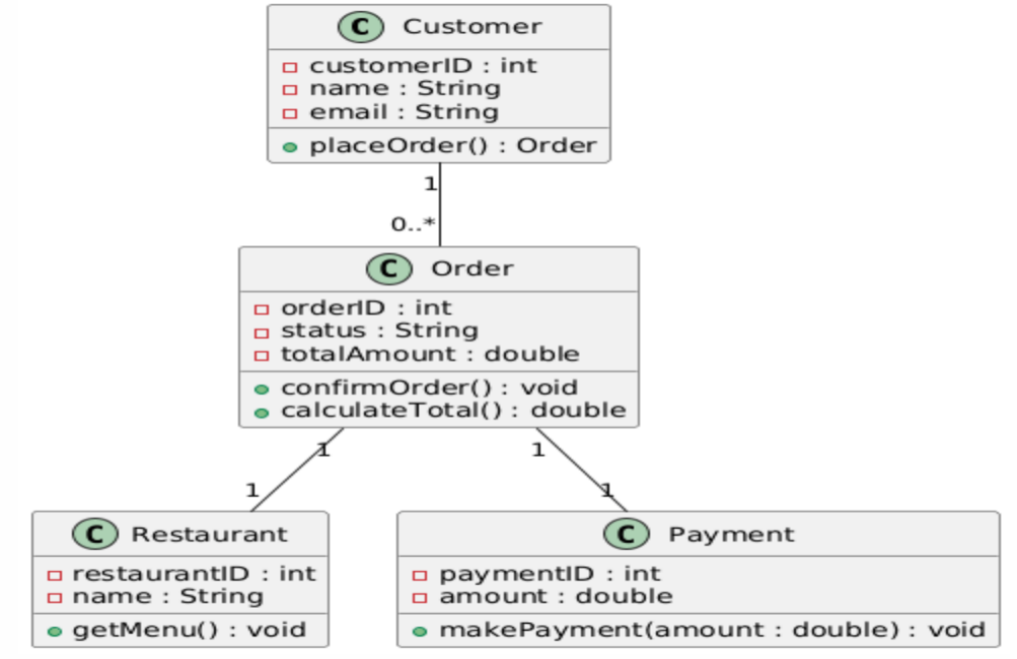
Part 1 – Design Class Diagram (DCD)

A Design Class Diagram (DCD) is a structural UML diagram used in the design phase of software development. It represents software classes, their attributes, operations, and relationships. Unlike a conceptual class diagram, a DCD includes technical details such as data types, method signatures, visibility, and navigability.

A DCD is derived from the domain model and refined using sequence diagrams. The messages exchanged in sequence diagrams become operations in the classes of the DCD. Associations in the DCD represent object references needed to support system behavior.

Design Class Diagram – Stomach Saver

The following Design Class Diagram represents the main software classes of the Stomach Saver online food ordering system. The diagram is based on the existing domain model and the sequence diagrams created in previous weeks. It includes customers placing orders, restaurants providing menus, and payments being processed.



Week9_individual_assignment_Sulaiman (Stan) Abu Romman

Part 2 – Individual Research Task: GRASP

GRASP stands for General Responsibility Assignment Software Patterns. It is a set of guidelines used in object-oriented design to assign responsibilities to classes in a clear and maintainable way. GRASP helps developers decide which class should handle which responsibility based on principles rather than intuition.

The main GRASP techniques include Information Expert, Creator, Controller, Low Coupling, High Cohesion, Polymorphism, Indirection, and Pure Fabrication. For example, the Information Expert principle states that responsibility should be assigned to the class that has the necessary information to fulfill it. The Controller pattern suggests assigning system event handling to a dedicated controller class.

The purpose of GRASP in software development is to improve software quality by promoting well-structured, understandable, and maintainable designs. By applying GRASP principles, developers reduce dependencies between classes, improve flexibility, and make systems easier to extend and maintain. GRASP is especially useful when transforming analysis models, such as use cases and sequence diagrams, into concrete design models like Design Class Diagrams.

Week10_group_and_individual_assignment_Sulaiman (Stan) Abu Romman

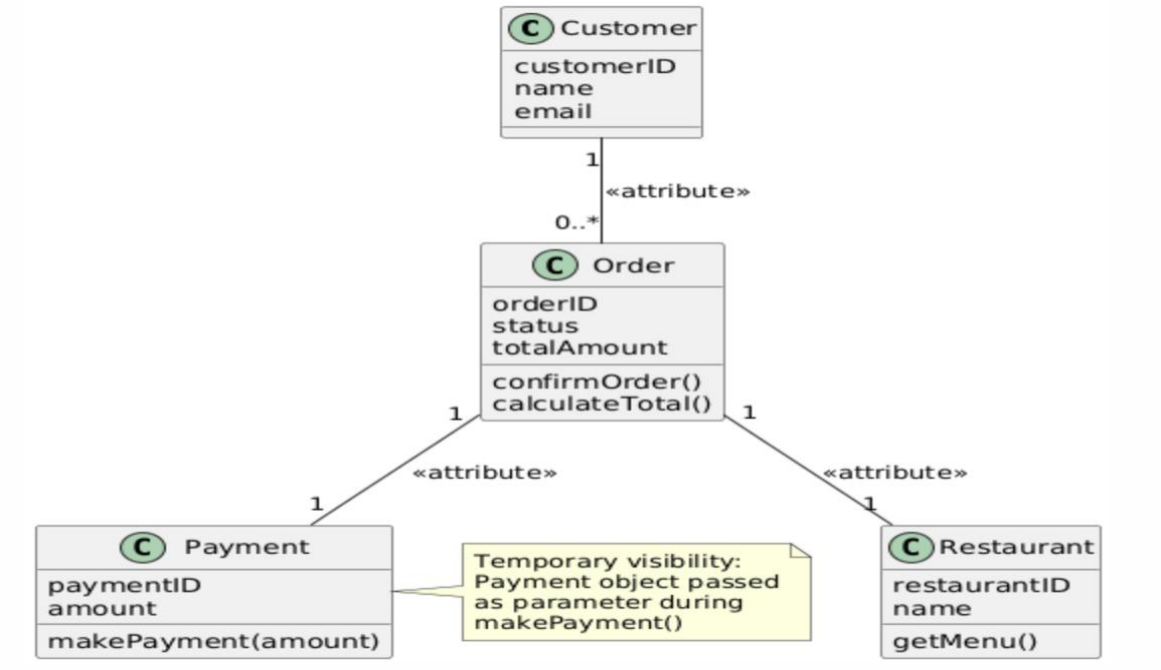
Part 1 – Group Assignment: Visibility (Design for Change)

Visibility refers to the ability of one object to know or access another object. In object-oriented design, visibility determines how objects communicate and how strongly they are coupled. Correct use of visibility helps reduce unnecessary dependencies and supports design for change.

In the Stomach Saver system, visibility is mainly achieved through attribute visibility and parameter visibility. Attribute visibility is used when objects need long-term access to each other, while parameter visibility is used when objects are temporarily passed during method execution. Global visibility is avoided to keep coupling low.

Expanded Design Class Diagram with Visibility – Stomach Saver

The following Design Class Diagram shows how visibility is applied in the Stomach Saver system. Associations represent attribute visibility, while method parameters represent temporary visibility.



Part 2 – Group Assignment: GRASP (Design for Change)

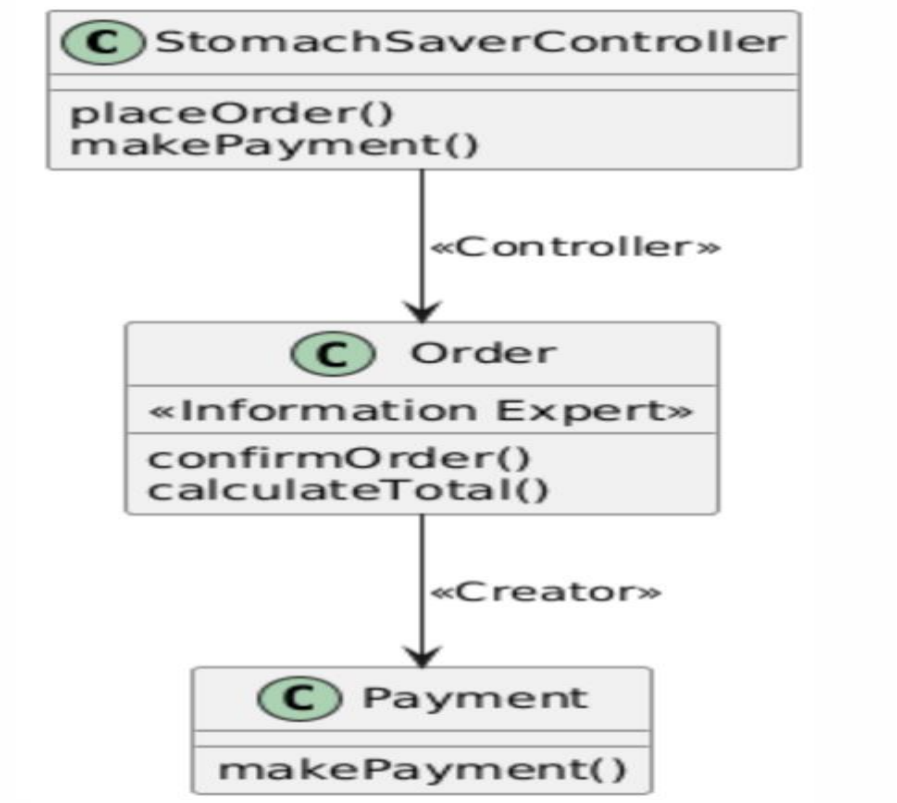
GRASP stands for General Responsibility Assignment Software Patterns. It provides principles for assigning responsibilities to classes in a way that results in low coupling and high cohesion.

In the Stomach Saver system, the Information Expert principle is applied by assigning the responsibility of calculating the total amount to the Order class, since it contains the required data. The Creator principle is applied by letting the Order class create Payment objects, as it closely manages and uses them.

The Controller pattern can be applied by introducing a controller that receives system events from the user interface and delegates responsibilities to domain classes such as Order and Payment. This prevents bloated domain objects and keeps responsibilities well separated.

Applying GRASP principles ensures that the Design Class Diagram remains flexible and supports future changes without major restructuring.

GRASP Applied to Stomach Saver – Visual Overview



Part 3 – Individual Assignment: Abstract Classes and Interfaces in a DCD

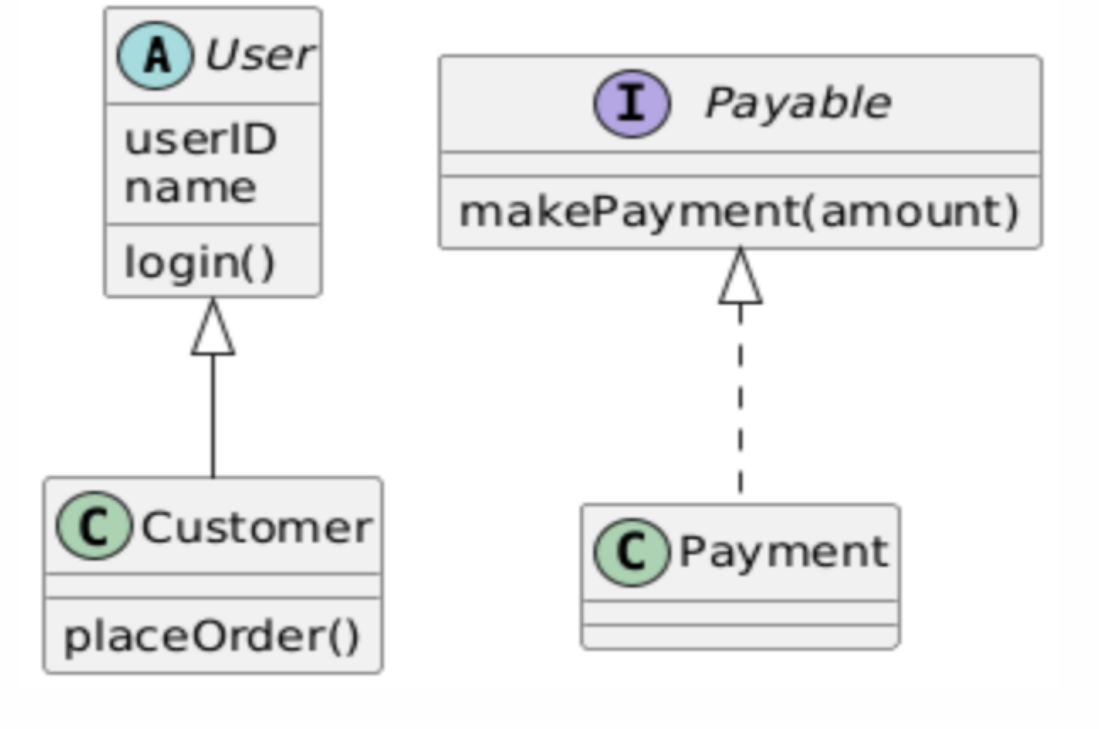
An abstract class is a class that cannot be instantiated and is used to define common behavior for its subclasses. It may contain both abstract and concrete methods. Abstract classes promote code reuse and shared structure.

An interface defines a set of methods that a class must implement without specifying how those methods are implemented. Interfaces are used to define roles and support flexible designs by reducing dependency on concrete classes.

In a Design Class Diagram, abstract classes are typically marked as abstract, and interfaces are shown using the «interface» stereotype. These techniques help separate general behavior from specific implementations.

The purpose of abstract classes and interfaces is to design systems that are easy to extend and modify. By relying on abstractions instead of concrete implementations, systems such as Stomach Saver can evolve without breaking existing functionality.

Visual representation – Abstract Class and Interface Example



Week11_group_and_individual_assignment_Sulaiman (Stan) Abu Romman

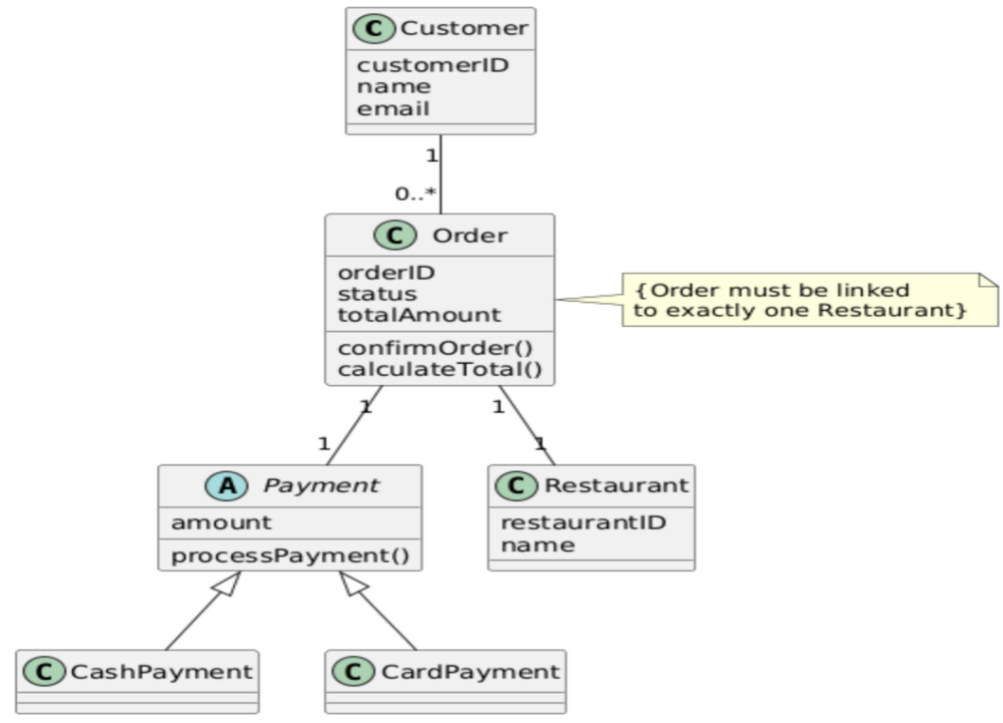
Part 1 – Group Assignment: Fine Tune Your Design Class Diagram

In this assignment, the Design Class Diagram of the Stomach Saver system is reviewed and refined by applying advanced UML concepts. Fine-tuning the DCD improves clarity, correctness, and prepares the design for implementation. The refinements include the use of constraints, stereotypes, inheritance, and polymorphism.

Constraints are used to express logical rules that must always be respected by the system. In the Stomach Saver system, constraints ensure valid relationships between orders, payments, and restaurants. Stereotypes are applied to clarify the role of classes and relationships and to extend standard UML notation.

Inheritance and polymorphism are used where shared behavior exists. This supports the Open–Closed Principle by allowing the system to be extended without modifying existing classes. The refined DCD reflects a more flexible and maintainable design.

Refined Design Class Diagram – Stomach Saver



Part 2 – Individual Assignment: Reflection on Fine-Tuning the DCD

Fine-tuning the Design Class Diagram improves the overall quality of the system design. By introducing abstract classes, inheritance, and constraints, the diagram becomes more expressive and closer to a real implementation. The use of an abstract Payment class allows different payment methods to be added without changing the Order class, which supports polymorphism and design for change.

Constraints help document business rules directly in the model, reducing ambiguity. Stereotypes and refined relationships make the diagram easier to understand and maintain. These improvements ensure that the DCD accurately represents system behavior and is consistent with object-oriented design principles such as low coupling, high cohesion, and the Open–Closed Principle.